Name of Student: Minisha Singh	**PRN: 25070122292**
Semester: III	**Year: AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No: 25	
TITLE: Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.	

Problem Statement

Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Learning Objectives

To create and manage multiple threads using the Thread class.

Programme

1.Create a program where multiple threads print numbers simultaneously

Code:

```
class NumberThread extends Thread {
   String threadName;

   NumberThread(String name) {
      threadName = name;
   }

   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println(threadName + " : " + i);

         try {
            Thread.sleep(500);
         } catch (InterruptedException e) {
            System.out.println(e);
         }
      }
   }
```

```
}

public class MultipleThreads {
    public static void main(String[] args) {
        NumberThread t1 = new NumberThread("Thread 1");
        NumberThread t2 = new NumberThread("Thread 2");
        NumberThread t3 = new NumberThread("Thread 3");

        System.out.println("Thread 1 State: " + t1.getState());

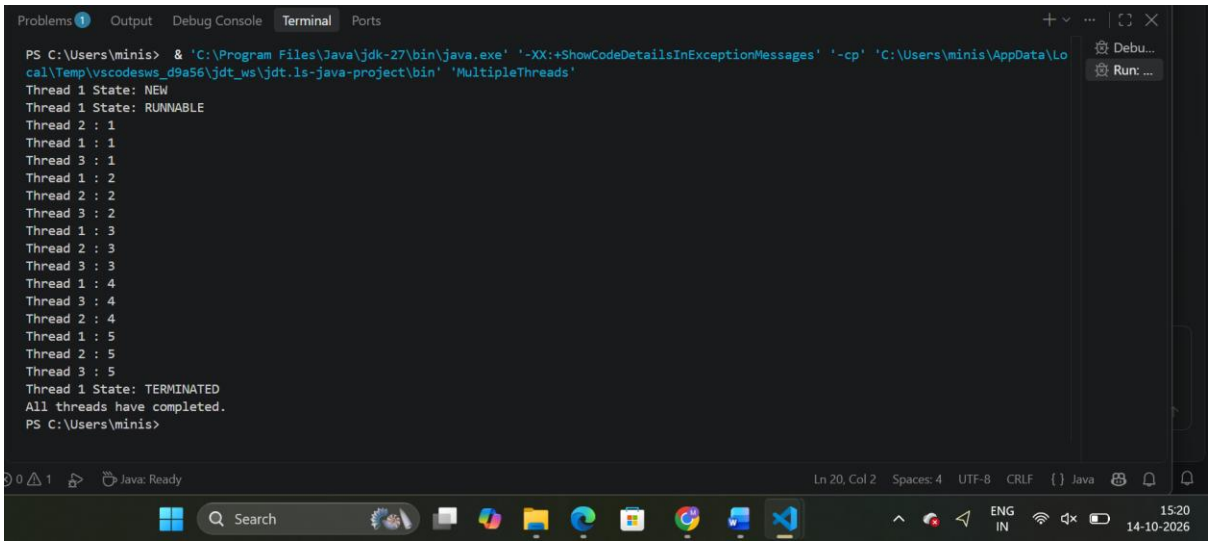
        t1.start();
        t2.start();
        t3.start();

        System.out.println("Thread 1 State: " + t1.getState());

        try {
            t1.join();
            t2.join();
            t3.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("Thread 1 State: " + t1.getState());
        System.out.println("All threads have completed.");
    }
}
```

Output:



2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

Code:

```
class BankAccount {
   int balance = 1000;

   synchronized void deposit(int amount) {
      balance += amount;
      System.out.println("Deposited: Rs. " + amount);
      System.out.println("Balance after deposit: Rs. " + balance);
   }

   synchronized void withdraw(int amount) {
      if (balance >= amount) {
         balance -= amount;
         System.out.println("Withdrawn: Rs. " + amount);
         System.out.println("Balance after withdrawal: Rs. " + balance);
      } else {
         System.out.println("Insufficient balance!");
      }
   }
}

class DepositThread extends Thread {
   BankAccount account;

   DepositThread(BankAccount account) {
      this.account = account;
   }

   public void run() {
      account.deposit(500);
      account.deposit(300);
   }
}

class WithdrawThread extends Thread {
   BankAccount account;

   WithdrawThread(BankAccount account) {
      this.account = account;
   }
```

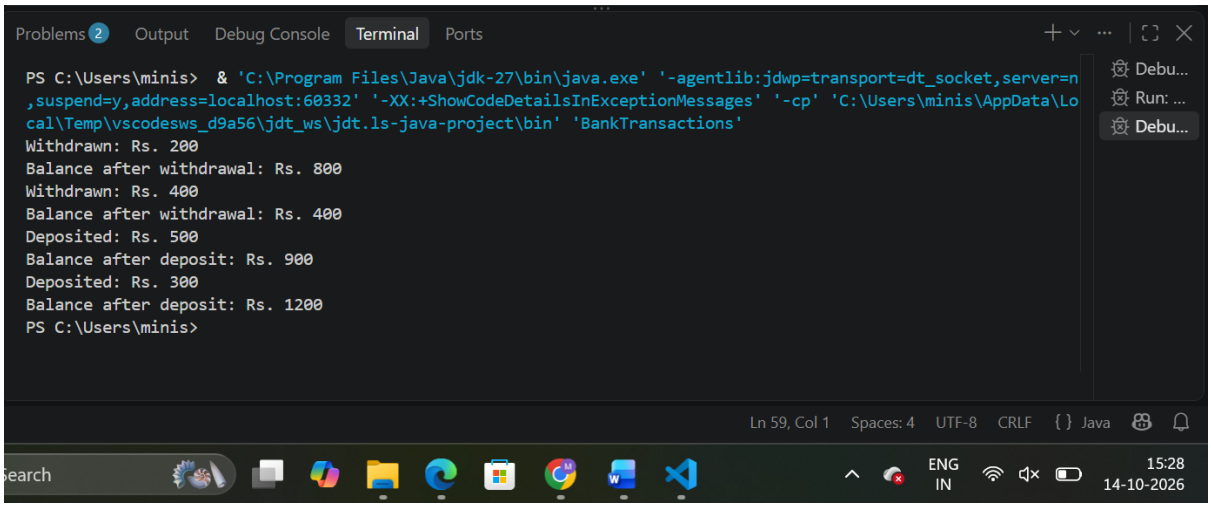
```
    public void run() {
        account.withdraw(200);
        account.withdraw(400);
    }
}

public class BankTransactions {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new WithdrawThread(account);

        t1.start();
        t2.start();
    }
}
```

Output:



Conclusion

Thus, we learned how to create and manage multiple threads using the Thread class in Java. We understood how threads can execute different tasks simultaneously. We also learned how to start threads using the start() method and execute their tasks using the run() method. The program demonstrated how multiple threads can work independently. This helped us understand the basic concept of multithreading in Java.